

Part 1: Handling missing values & Basic concepts in Probability

```
In [20]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
In [21]: df = pd.read_csv('Cars.csv')
print("Dataset Dimensions (rows, columns): ", df.shape, '\n')
print("Data Sample: ", '\n', df.head(), '\n')
```

Dataset Dimensions (rows, columns): (205, 24)

Data Sample:

	carname	fueltype	aspiration	doornumber	car
body \					
0	alfa-romero giulia	gas	std	two	converti
ble					
1	alfa-romero stelvio	gas	std	two	converti
ble					
2	alfa-romero Quadrifoglio	gas	std	two	hatchb
ack					
3	audi 100 ls	gas	std	four	se
dan					
4	audi 100ls	gas	std	four	se
dan					

	drivewheel	engineloation	wheelbase	carlength	carwidth	...	en
ginesize \							
0	rwd	front	88.6	168.8	64.1	...	
130.0							
1	rwd	front	88.6	168.8	64.1	...	
130.0							
2	rwd	front	94.5	171.2	65.5	...	
152.0							
3	fwd	front	99.8	176.6	66.2	...	
109.0							
4	4wd	front	99.4	176.6	66.4	...	
136.0							

	fuelsystem	boreratio	stroke	compressionratio	horsepower	peakrpm
citympg \						
0	mpfi	3.47	2.68	9.0	111	5000
21						
1	mpfi	3.47	2.68	9.0	111	5000
21						
2	mpfi	2.68	3.47	9.0	154	5000
19						
3	mpfi	3.19	3.40	10.0	102	5500
24						
4	mpfi	3.19	3.40	8.0	115	5500
18						

	highwaympg	price
0	27	13495.0
1	27	16500.0
2	26	16500.0
3	30	13950.0
4	22	17450.0

[5 rows x 24 columns]

```
In [22]: print("Description for qualitative data: ", '\n', df.describe(include=
print("Description for quantitative data: ", '\n', df.describe(), ')
```

Description for qualitative data:

	carname	fueltype	aspiration	doornumber	carbody	drivew
heel \						
count	205	205	205	205	205	
unique	147	2	2	2	5	
top	toyota corona	gas	std	four	sedan	
fwd						
freq	6	185	168	115	96	
120						

	enginelocation	engine	cylindernumber	fuelsystem
count	205	205	205	205
unique	2	7	7	8
top	front	ohc	four	mpfi
freq	202	148	159	94

Description for quantitative data:

	wheelbase	carlength	carwidth	carheight	curbweight
\					
count	194.000000	205.000000	185.000000	190.000000	205.000000
mean	98.669588	174.049268	65.932973	53.738947	2555.565854
std	5.934168	12.337289	2.113143	2.470452	520.680204
min	86.600000	141.100000	60.300000	47.800000	1488.000000
25%	94.500000	166.300000	64.200000	52.000000	2145.000000
50%	97.000000	173.200000	65.500000	54.100000	2414.000000
75%	101.800000	183.100000	66.600000	55.575000	2935.000000
max	120.900000	208.100000	72.300000	59.800000	4066.000000

	enginesize	boreratio	stroke	compressionratio	horsepo
wer \					
count	185.000000	205.000000	205.000000	205.000000	205.000
mean	124.081081	3.329756	3.255415	10.142537	104.117
std	38.528771	0.270844	0.313597	3.972040	39.544
min	61.000000	2.540000	2.070000	7.000000	48.000
25%	97.000000	3.150000	3.110000	8.600000	70.000
50%	110.000000	3.310000	3.290000	9.000000	95.000
75%	141.000000	3.580000	3.410000	9.400000	116.000
max	308.000000	3.940000	4.170000	23.000000	288.000

	peakrpm	citympg	highwaympg	price
count	205.000000	205.000000	205.000000	205.000000
mean	5125.121951	25.219512	30.751220	13276.710571
std	476.985643	6.542142	6.886443	7988.852332
min	4150.000000	13.000000	16.000000	5118.000000
25%	4800.000000	19.000000	25.000000	7788.000000
50%	5200.000000	24.000000	30.000000	10295.000000
75%	5500.000000	30.000000	34.000000	16503.000000

max 6600.000000 49.000000 54.000000 45400.000000

```
In [23]: # Q1: How many null values are there in each column?
print("Null values in each column: ", '\n', df.isnull().sum(), '\n')
```

Null values in each column:

```
carname      0
fueltype     0
aspiration   0
doornumber   0
carbody      0
drivewheel   0
engineloction 0
wheelbase    11
carlength    0
carwidth     20
carheight    15
curbweight   0
enginetype   0
cylindernumber 0
enginesize   20
fuelsystem   0
boreratio    0
stroke       0
compressionratio 0
horsepower   0
peakrpm      0
citympg      0
highwaympg   0
price        0
dtype: int64
```

```
In [24]: # Q1_1: Replace the null value of column "wheelbase" with the max v
max_wheelbase = df['wheelbase'].max()
df['wheelbase'].fillna(max_wheelbase, inplace=True)
print("Null values in 'wheelbase' column after replacement: ", df['wheelbase'].isnull().sum())
```

Null values in 'wheelbase' column after replacement: 0

```
In [25]: # Q1_2: Replace the null value of column "carwidth" with the Min va
min_carwidth = df['carwidth'].min()
df['carwidth'].fillna(min_carwidth, inplace=True)
print("Null values in 'carwidth' column after replacement: ", df['carwidth'].isnull().sum())
```

Null values in 'carwidth' column after replacement: 0

```
In [26]: # Q1_3: Use Linear Interpolation to change the null value of column
df['carheight'] = df['carheight'].interpolate(method='linear')
print("Null values in 'carheight' after interpolation:", df['carheight'].isnull().sum())
```

Null values in 'carheight' after interpolation: 0

```
In [27]: # Q1_4: Replace the null value of column "enginesize" with the mod
mod_enginesize = df['enginesize'].mode()[0]
```

```
df['enginesize'].fillna(mod_enginesize, inplace=True)
print("Null values in 'enginesize' column after replacement: ", df[
```

Null values in 'enginesize' column after replacement: 0

```
In [28]: # Q2: What is the Probability that the Car is Sedan ? (1 mark)
total_cars = len(df)
sedan_cars = len(df[df['carbody'] == 'sedan'])
probability_sedan = sedan_cars / total_cars
print(f"Probability that the Car is Sedan: {probability_sedan}")
```

Probability that the Car is Sedan: 0.4682926829268293

```
In [29]: # Q3: What is the Probability that the car is Sedan and works with
sedan_with_gas = len(df[(df['carbody'] == 'sedan') & (df['fueltype'
probability_sedan_gas = sedan_with_gas / total_cars
print(f"Probability that the car is Sedan and works with gas: {prob
```

Probability that the car is Sedan and works with gas: 0.3951219512195122

```
In [30]: # Q4:What is the probability that a car is Sedan given it is a gas
gas_cars = len(df[df['fueltype'] == 'gas'])
probability_sedan_given_gas = sedan_with_gas / gas_cars
print(f"Probability that a car is Sedan given it is a gas fuel type
```

Probability that a car is Sedan given it is a gas fuel type: 0.43783783783783786

```
In [31]: # Q5: Given that the car height is higer than 50, find the probabil
height_greater_50 = df[df['carheight'] > 50]
height_and_engine = height_greater_50[height_greater_50['enginesize
probability_engine_given_height = len(height_and_engine) / len(heig
print(f"Probability that the car engine size is greater than 130 gi
```

Probability that the car engine size is greater than 130 given that the car height is greater than 50: 0.2916666666666667

Part 2: Probability Functions

```
In [39]: # Q6: Load dataset 'students.csv' and show first rows, shape, and c
df_students = pd.read_csv('students.csv')
print("Students Dataset Sample: ", '\n', df_students.head(), '\n')
print("Students Dataset Dimensions (rows, columns): ", df_students.

df_students_numeric = df_students.select_dtypes(exclude='object')
print("Skewness of the students dataset: ", '\n', df_students_numer
```

Students Dataset Sample:

	stud.id	name	gender	age	height	weight	religion \
0	833917	Gonzales, Christina	Female	19	160	64.8	Muslim
1	898539	Lozano, T'Hani	Female	19	172	73.0	Other
2	379678	Williams, Hanh	Female	22	168	70.6	Protestant
3	807564	Nem, Denzel	Male	19	183	79.7	Other
4	383291	Powell, Heather	Female	21	175	71.4	Catholic

	nc.score	semester	major	minor \
0	1.91	1st	Political Science	Social Sciences
1	1.56	2nd	Social Sciences	Mathematics and Statistics
2	1.24	3rd	Social Sciences	Mathematics and Statistics
3	1.37	2nd	Environmental Sciences	Mathematics and Statistics
4	1.46	1st	Environmental Sciences	Mathematics and Statistics

	score1	score2	online.tutorial	graduated	salary
0	NaN	NaN	0	0	NaN
1	NaN	NaN	0	0	NaN
2	45.0	46.0	0	0	NaN
3	NaN	NaN	0	0	NaN
4	NaN	NaN	0	0	NaN

Students Dataset Dimensions (rows, columns): (8239, 16)

Skewness of the students dataset:

stud.id	0.018612
age	4.427823
height	-0.009244
weight	0.651004
nc.score	0.446834
score1	-0.318819
score2	-0.304878
online.tutorial	0.467502
graduated	1.403900
salary	0.273835

dtype: float64

```
In [40]: # Q7: What is the probability that the salary is greater than 20000

# 1. Identify the distribution by
# Plotting Histogram
plt.hist(df_students['salary'], bins=20, edgecolor='black')
plt.title("Histogram of Salary")
```

```

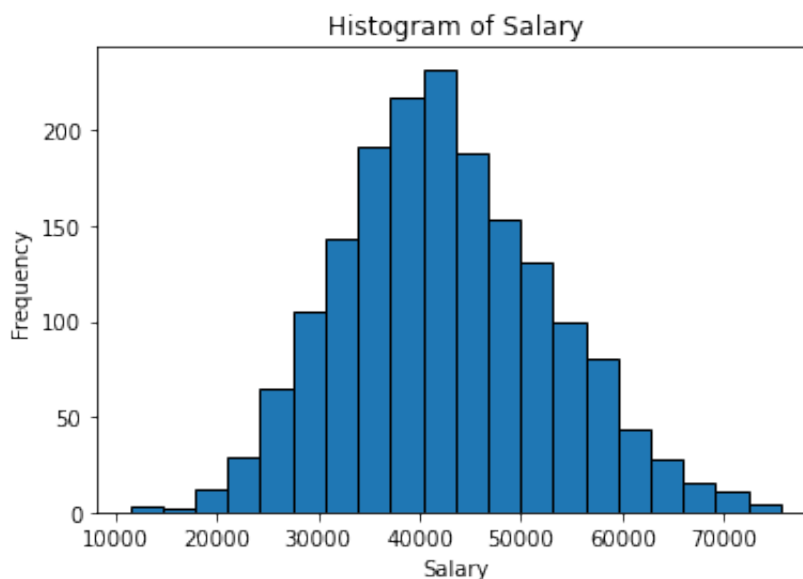
plt.xlabel("Salary")
plt.ylabel("Frequency")
plt.show()
# calculate the skewness
salary_skew = df_students['salary'].skew()
print("Skewness of salary:", salary_skew, '\n')
# Which distribution does this data follow?
print("The salary data appears to be right-skewed, suggesting it ma

# 2. Compute mean and std
mean_salary = df_students['salary'].mean()
std_salary = df_students['salary'].std()
print(f"Mean Salary: {mean_salary}, Standard Deviation: {std_salary}

# 3. Convert to Z-scores
lower_z = (20000 - mean_salary) / std_salary
upper_z = (30000 - mean_salary) / std_salary
print(f"Z-scores: lower = {lower_z}, upper = {upper_z} \n")

# 4. Use CDF difference with the use of the right distribution func
from scipy.stats import norm
probability = norm.cdf(upper_z) - norm.cdf(lower_z)
print(f"Probability that salary is between 20000 and 30000: {probab

```



Skewness of salary: 0.27383514161440997

The salary data appears to be right-skewed, suggesting it may follow a log-normal distribution or a similar distribution with a long tail on the right side.

Mean Salary: 42522.112364181354, Standard Deviation: 10333.139905547341

Z-scores: lower = -2.1796000605866537, upper = -1.2118400097785247

Probability that salary is between 20000 and 30000: 0.09814325462173745

In [41]: # Q8: A factory produces 200 chips with defect rate 8%. Answer the
1. What is the distribution name of this problem, based on the fo

```

print("This problem follows a Binomial distribution, as it involves

# 2. What is the probability of getting exactly 14 defective chips
from scipy.stats import binom
n = 200 # number of trials
p = 0.08 # probability of success (defective chip)
k = 14 # number of defective chips
probability_14_defective = binom.pmf(k, n, p)
print(f"Probability of getting exactly 14 defective chips: {probabi

# 3. At least 14 defective ?

probability_at_least_14 = 1 - binom.cdf(13, n, p)
print(f"Probability of getting at least 14 defective chips: {probab

# 4. At most 15 defective ?
probability_at_most_15 = binom.cdf(15, n, p)
print(f"Probability of getting at most 15 defective chips: {probabi

# 5. Expected defects and standard deviation
expected_defects = n * p
std_defects = (n * p * (1 - p))*0.5
print(f"Expected number of defective chips: {expected_defects}")
print(f"Standard deviation of defective chips: {std_defects}")

```

This problem follows a Binomial distribution, as it involves a fixed number of independent trials (200 chips) with two possible outcomes (defective or not defective) and a constant probability of success (defect rate of 8%).

Probability of getting exactly 14 defective chips: 0.0954115649226197

Probability of getting at least 14 defective chips: 0.7357078859493741

Probability of getting at most 15 defective chips: 0.4625822359332888

Expected number of defective chips: 16.0

Standard deviation of defective chips: 3.8366652186501757